Rapport de projet : Segmentation automatique d'IRMs cardiaques par Deep Learning

Réalisé par : Ghizlane ECH-CHIGUER

Encadré par : Omar AIT AIDER

Année académique : 2024/2025

1. Introduction

La segmentation des IRMs cardiaques joue un rôle essentiel dans l'analyse des structures anatomiques du cœur, notamment l'endocarde et l'épicarde. Ce projet vise à mettre en place un modèle de **segmentation automatique** en utilisant un réseau de neurones convolutionnel (**CNN**) de type **U-Net**. L'objectif principal est de remplacer les méthodes manuelles coûteuses en temps par une approche automatisée capable de produire des résultats précis.
L'étude repose sur :

- **Annotation manuelle** d'images IRM 2D.
- **Entraînement d'un modèle U-Net** avec TensorFlow/Keras.
- **Évaluation des performances** par des métriques comme le coefficient **Dice**.

2. Méthodologie

2.1 Base de Données

Les données sont des **images IRM 2D** au format `.png`, annotées pour segmenter l'endocarde. La structure est organisée comme suit :

- **Images d'entrée** : `/frames`
- **Masques annotés :** `/masks endo`

2.2 Prétraitement des Données

Pour préparer les données à l'entraînement, les étapes suivantes ont été réalisées :

a. Normalisation et Augmentation des Données

- Les pixels des images ont été **normalisés** entre 0 et 1 pour homogénéiser les données.
- Des **transformations géométriques** ont été appliquées pour enrichir la base d'entraînement (data augmentation).

Code d'augmentation des données :

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range =10,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True)

val_datagen = ImageDataGenerator(
    rescale=1./255)
```

b. Générateurs de Données

Les images et masques sont chargés en parallèle grâce à `flow_from_directory` :

Extrait du code :

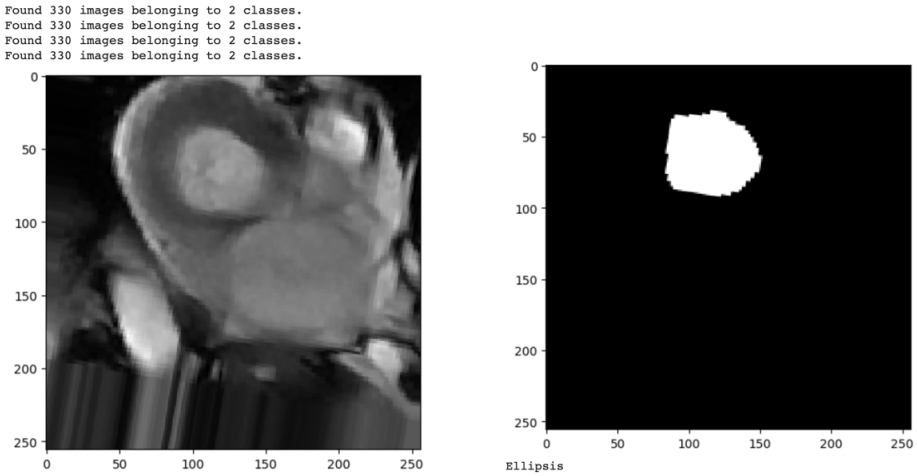
```
# Output = train_image_generator (images from TRAIN_FRAME_PATH)
train_image_generator = train_datagen.flow_from_directory(
    TRAIN_FRAME_PATH,
    target_size=(256,256),
    batch_size=BATCH_SIZE,
    class_mode=None,
    color_mode='grayscale',
    seed=123)

# The images must be found in a subdir of TRAIN_MASK_PATH for instance
/Train
# Use the same seed as training images so that augmentation is the same
train_mask_generator =
train_datagen.flow_from_directory(TRAIN_MASK_PATH,
target_size=(256,256),
batch_size=BATCH_SIZE,
class_mode=None,
color_mode='grayscale',
seed=123)

# The images must be found in a subdir of VAL_FRAME_PATH for instance
/Val
# Output = val_image_generator (images from VAL_FRAME_PATH)
val_image_generator = val_datagen.flow_from_directory(VAL_FRAME_PATH,
target_size=(256,256),
batch_size=BATCH_SIZE,
class_mode=None,
color_mode='grayscale',
seed=123)

# The images must be found in a subdir of VAL_MASK_PATH for instance
/Val
# Use the same seed as validation images so that aumentation is the
same
val_mask_generator = val_datagen.flow_from_directory(VAL_MASK_PATH,
target_size=(256,256),
batch_size=BATCH_SIZE,
class_mode=None,
color_mode='grayscale', seed=123)
```

Les générateurs d'images et de masques sont zippés pour produire des paires `(image, masque)` :

Extrait du code:

```
train_generator = zip(train_image_generator, train_mask_generator)
val_generator = zip(val_image_generator, val_mask_generator)
```

À l'issue de l'exécution de la cellule du code , les résultats obtenus sont les suivants :



2.3 Architecture du Modèle U-Net

Le modèle U-Net se compose de trois parties principales :

1. **Encodeur (Downsampling)** : Extraction des caractéristiques avec `Conv2D` et réduction de la résolution par `MaxPooling2D`.
2. **Bottleneck** : Couche centrale capturant les informations les plus importantes.
3. **Décodeur (Upsampling)** : Reconstruction des images segmentées avec `Conv2DTranspose` et concaténation.

Code complet :

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, concatenate, Conv2D,
MaxPooling2D, Conv2DTranspose

def get_UNet(img_rows, img_cols):
    inputs = Input((img_rows, img_cols, 1))
    conv1 = Conv2D(32, (3, 3), activation='relu',
padding='same')(inputs)
    conv1 = Conv2D(32, (3, 3), activation='relu',
padding='same')(conv1)
    pool1 = MaxPooling2D(pool_size=(2, 2))(conv1)
```

```
    #Partie encodeur
    conv2 = Conv2D(64, (3, 3), activation='relu',
padding='same')(pool1)
    conv2 = Conv2D(64, (3, 3), activation='relu',
padding='same')(conv2)
    pool2 = MaxPooling2D(pool_size=(2, 2))(conv2)

    conv3 = Conv2D(128, (3, 3), activation='relu',
padding='same')(pool2)
    conv3 = Conv2D(128, (3, 3), activation='relu',
padding='same')(conv3)
    pool3 = MaxPooling2D(pool_size=(2, 2))(conv3)

    conv4 = Conv2D(256, (3, 3), activation='relu',
padding='same')(pool3)
    conv4 = Conv2D(256, (3, 3), activation='relu',
padding='same')(conv4)
    pool4 = MaxPooling2D(pool_size=(2, 2))(conv4)


    conv5 = Conv2D(512, (3, 3), activation='relu',
padding='same')(pool4)
    conv5 = Conv2D(512, (3, 3), activation='relu',
padding='same')(conv5)

    up6 = concatenate([Conv2DTranspose(256, (2, 2), strides=(2, 2),
padding='same')(conv5), conv4], axis=3)
    conv6 = Conv2D(256, (3, 3), activation='relu', padding='same')(up6)
    conv6 = Conv2D(256, (3, 3), activation='relu',
padding='same')(conv6)

    up7 = concatenate([Conv2DTranspose(128, (2, 2), strides=(2, 2),
padding='same')(conv6), conv3], axis=3)
    conv7 = Conv2D(128, (3, 3), activation='relu', padding='same')(up7)
    conv7 = Conv2D(128, (3, 3), activation='relu',
padding='same')(conv7)

    up8 = concatenate([Conv2DTranspose(64, (2, 2), strides=(2, 2),
padding='same')(conv7), conv2], axis=3)
    conv8 = Conv2D(64, (3, 3), activation='relu', padding='same')(up8)
    conv8 = Conv2D(64, (3, 3), activation='relu',
padding='same')(conv8)

    up9 = concatenate([Conv2DTranspose(32, (2, 2), strides=(2, 2),
padding='same')(conv8), conv1], axis=3)
    conv9 = Conv2D(32, (3, 3), activation='relu', padding='same')(up9)
    conv9 = Conv2D(32, (3, 3), activation='relu',
padding='same')(conv9)
```

```
    conv10 = Conv2D(1, (1, 1), activation='sigmoid')(conv9)

    model = Model(inputs=[inputs], outputs=[conv10])

    return model
```

2.4 Fonction de Perte et Compilation

Une fonction de perte **combinée** entre **Dice Loss** et **Binary Crossentropy** a été utilisée :

```
def combined_loss(y_true, y_pred, dice_weight=0.5, bce_weight=0.5):
    dice = dice_loss(y_true, y_pred)
    bce = tf.keras.losses.BinaryCrossentropy()(y_true, y_pred)
    return dice_weight * dice + bce_weight * bce
```

Le modèle est compilé avec :

- **Optimiseur** : Adam.
- **Métriques** : Dice Coefficient et précision.

```
m.compile(optimizer=opt,
          loss=combined_loss,  # Fonction de perte combinée
          metrics=[dice_coefficient, 'accuracy'])  # Ajout du Dice
coefficient comme métrique
```

2.5 Entraînement du Modèle

L'entraînement du modèle a été effectué sur **50 epochs** avec des **callbacks** pour surveiller la performance :

- **ModelCheckpoint** : Sauvegarde des meilleurs poids.
- **EarlyStopping** : Arrêt si la validation ne s'améliore plus.

Code :

```
# Paramètres d'entraînement
NO_OF_EPOCHS = 50
steps_per_epoch = len(train_image_generator)
validation_steps = len(val_image_generator)

# Fonction génératrice pour l'entraînement
def train_generator_fn():
    while True:
        img, mask = next(train_generator)
        yield img, mask

# Fonction génératrice pour la validation
```

```python
def val_generator_fn():
    while True:
        img, mask = next(val_generator)
        yield img, mask

# Entraîner le modèle
results = m.fit(
    train_generator_fn(),
    steps_per_epoch=steps_per_epoch,
    epochs=NO_OF_EPOCHS,
    validation_data=val_generator_fn(),
    validation_steps=validation_steps,
    callbacks=callbacks_list
)
```

À l'issue de l'exécution de la cellule du code , les résultats obtenus sont les suivants :

```
Epoch 45: val_loss improved from 0.18837 to 0.18802, saving model to ./weights.keras
11/11 ━━━━━━━━━━━━━━━━━━━━ 10s 990ms/step - accuracy: 0.9433 - dice_coefficient: 0.6474 - loss: 0.2674 - val_accuracy: 0.9682 - val_dice_coefficient: 0.7185
Epoch 46/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 500ms/step - accuracy: 0.9327 - dice_coefficient: 0.5775 - loss: 0.3185
Epoch 46: val_loss did not improve from 0.18802
11/11 ━━━━━━━━━━━━━━━━━━━━ 10s 985ms/step - accuracy: 0.9312 - dice_coefficient: 0.5717 - loss: 0.3239 - val_accuracy: 0.9502 - val_dice_coefficient: 0.6065
Epoch 47/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 495ms/step - accuracy: 0.9274 - dice_coefficient: 0.5149 - loss: 0.3431
Epoch 47: val_loss did not improve from 0.18802
11/11 ━━━━━━━━━━━━━━━━━━━━ 11s 1s/step - accuracy: 0.9266 - dice_coefficient: 0.5149 - loss: 0.3442 - val_accuracy: 0.9626 - val_dice_coefficient: 0.6815 -
Epoch 48/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 494ms/step - accuracy: 0.9416 - dice_coefficient: 0.6211 - loss: 0.2771
Epoch 48: val_loss did not improve from 0.18802
11/11 ━━━━━━━━━━━━━━━━━━━━ 10s 896ms/step - accuracy: 0.9403 - dice_coefficient: 0.6170 - loss: 0.2815 - val_accuracy: 0.9628 - val_dice_coefficient: 0.6943
Epoch 49/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 501ms/step - accuracy: 0.9446 - dice_coefficient: 0.6225 - loss: 0.2690
Epoch 49: val_loss improved from 0.18802 to 0.15644, saving model to ./weights.keras
11/11 ━━━━━━━━━━━━━━━━━━━━ 11s 1s/step - accuracy: 0.9440 - dice_coefficient: 0.6207 - loss: 0.2714 - val_accuracy: 0.9718 - val_dice_coefficient: 0.7709 -
Epoch 50/50
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 497ms/step - accuracy: 0.9325 - dice_coefficient: 0.6190 - loss: 0.3014
Epoch 50: val_loss improved from 0.15644 to 0.15054, saving model to ./weights.keras
11/11 ━━━━━━━━━━━━━━━━━━━━ 11s 1s/step - accuracy: 0.9317 - dice_coefficient: 0.6162 - loss: 0.3045 - val_accuracy: 0.9744 - val_dice_coefficient: 0.7752 -
```

3. Prédiction avec le modèle U-Net

Dans cette section, nous avons utilisé le modèle U-Net pour effectuer des prédictions sur les images d'entraînement et de validation. Le processus de prédiction a été divisé en plusieurs étapes : chargement du modèle, prétraitement des images, génération des prédictions et évaluation des résultats.

3.1. Chargement du modèle et des poids

Le modèle U-Net est initialement chargé à partir du module personnalisé `model` via la fonction `get_UNet()`. Ensuite, les poids optimisés du modèle, stockés dans le fichier `My_weights.weights.h5`, sont chargés pour permettre au modèle d'effectuer des prédictions sur les images.

Code :

```python
# Load model with optimized weights
m = model.get_UNet(img_rows=256, img_cols=256)

# Load weights
```

```
m.load_weights(WEIGHTS_PATH + '/My_weights.weights.h5')
```

3.2. Prétraitement des images

Les images d'entraînement et de validation sont chargées et redimensionnées à une taille uniforme de 256x256 pixels. Ces images sont ensuite normalisées pour que leurs valeurs soient comprises entre 0 et 1, facilitant ainsi le processus d'entraînement et de prédiction.

Code :

```
IMG_WIDTH = 256
IMG_HEIGHT = 256

train_frames = os.listdir(TRAIN_FRAME_PATH)
img_train = np.zeros((len(train_frames), IMG_HEIGHT, IMG_WIDTH, 1))

n = 0
for frame in train_frames:
    img = imread(TRAIN_FRAME_PATH + '/' + frame)
    img = resize(img, (IMG_HEIGHT, IMG_WIDTH))
    img_train[n, :, :, 0] = img
    n = n + 1

# Build a 4D numpy matrix to store all validation images from
VAL_FRAME_PATH
# output = img_val with indices 0 for image number, 1 ordinate, 2
abscissa, 3 channel (0 for gray level images)
val_frames = os.listdir(VAL_FRAME_PATH)
img_val = np.zeros((len(val_frames), IMG_HEIGHT, IMG_WIDTH, 1))

n = 0
for frame in val_frames:
    img = imread(VAL_FRAME_PATH + '/' + frame)
    img = resize(img, (IMG_HEIGHT, IMG_WIDTH))
    img_val[n, :, :, 0] = img
    n = n + 1
```

3.3. Prédictions sur les images d'entraînement et de validation

Une fois que les images sont prêtes, nous effectuons des prédictions pour les ensembles d'entraînement et de validation en utilisant la méthode `predict()` du modèle. Les résultats sont stockés dans les tableaux `pred_train` et `pred_val` respectivement.

Code :

```
# Prediction from model m
print("Predicting training images ...")
pred_train = m.predict(img_train)
```

```
print("Predicting validation images ...")
pred_val = m.predict(img_val)

print(f"Shape of predicted training images: {pred_train.shape}")
print(f"Shape of predicted validation images: {pred_val.shape}")
```

À l'issue de l'exécution de la cellule du code , les résultats obtenus sont les suivants :

```
Predicting training images ...
10/10 ━━━━━━━━━━━━━━━━━━━━ 172s 17s/step
Predicting validation images ...
1/1 ━━━━━━━━━━━━━━━━━━━━ 17s 17s/step
Shape of predicted training images: (300, 256, 256, 1)
Shape of predicted validation images: (30, 256, 256, 1)
```

3.4. Fusion des images d'entrée et des masques prédits

Une fois les prédictions effectuées, nous procédons à la fusion des images d'entraînement et de validation avec leurs masques prédits afin de visualiser les résultats de manière plus claire. Cette fusion consiste à superposer les masques prédits sur les images d'entrée, ce qui permet d'observer l'efficacité du modèle à détecter les caractéristiques des images.

Code:

```
# Fusion between initial images and predicted masks for display purpose
display_train = np.zeros((len(train_frames), IMG_HEIGHT, IMG_WIDTH, 3))
display_val = np.zeros((len(val_frames), IMG_HEIGHT, IMG_WIDTH, 3))

# Paths to save predicted images
TRAIN_PRED_PATH = DATA_PATH + '/train_preds'
if not os.path.isdir(TRAIN_PRED_PATH):
    os.makedirs(TRAIN_PRED_PATH)
VAL_PRED_PATH = DATA_PATH + '/val_preds'
if not os.path.isdir(VAL_PRED_PATH):
    os.makedirs(VAL_PRED_PATH)

# Charger et prétraiter les images
def preprocess_images(frame_path, frame_list):
    images = []
    for frame in frame_list:
        img = imread(os.path.join(frame_path, frame), as_gray=True)  #
Charger en niveaux de gris
        img = resize(img, (IMG_HEIGHT, IMG_WIDTH), mode='constant',
preserve_range=True)  # Redimensionner
        images.append(img / 255.0)  # Normaliser entre 0 et 1
    return np.expand_dims(np.array(images), axis=-1)  # Ajouter un axe
pour le canal

# Charger les masks
```

```
def preprocess_masks(mask_path, mask_list):
    masks = []
    for mask in mask_list:
        img = imread(os.path.join(mask_path, mask), as_gray=True)  #
Charger en niveaux de gris
        img = resize(img, (IMG_HEIGHT, IMG_WIDTH), mode='constant',
preserve_range=True)  # Redimensionner
        masks.append(img / 255.0)  # Normaliser entre 0 et 1
    return np.expand_dims(np.array(masks), axis=-1)  # Ajouter un axe
pour le canal

 # Paths to save masks
train_masks = preprocess_masks(TRAIN_MASK_PATH,
os.listdir(TRAIN_MASK_PATH))
val_masks = preprocess_masks(VAL_MASK_PATH, os.listdir(VAL_MASK_PATH))


# Charger les données d'entraînement et de validation
train_frames = preprocess_images(TRAIN_FRAME_PATH,
os.listdir(TRAIN_FRAME_PATH))
val_frames = preprocess_images(VAL_FRAME_PATH,
os.listdir(VAL_FRAME_PATH))

# Save the fusion of training images and corresponding predicted masks
to TRAIN_PRED_PATH
for i, frame in enumerate(train_frames):
    predicted_mask = m.predict(np.expand_dims(frame, axis=0))[0, :, :,
0]  # Prédire le masque
    predicted_mask_binary = (predicted_mask > 0.5).astype(np.uint8)  #
Appliquer un seuil binaire
    fused_image = np.stack((frame[:, :, 0], predicted_mask_binary *
255, frame[:, :, 0]), axis=-1)
    display_train[i] = fused_image
    save_path = os.path.join(TRAIN_PRED_PATH, f"train_fusion_{i}.png")
    imsave(save_path, fused_image.astype(np.uint8))

# Save the fusion of validation images and corresponding predicted
masks to VAL_PRED_PATH
for i, frame in enumerate(val_frames):
    predicted_mask = m.predict(np.expand_dims(frame, axis=0))[0, :, :,
0]
    predicted_mask_binary = (predicted_mask > 0.5).astype(np.uint8)
    fused_image = np.stack((frame[:, :, 0], predicted_mask_binary *
255, frame[:, :, 0]), axis=-1)
    display_val[i] = fused_image
    save_path = os.path.join(VAL_PRED_PATH, f"val_fusion_{i}.png")
    imsave(save_path, fused_image.astype(np.uint8))
```

Cette partie du code fait deux choses importantes :

1) **Fusion des images et des masques** : Les masques binaires prédits (0 ou 1) sont superposés sur les images d'origine pour obtenir une visualisation de la prédiction. Les images fusionnées sont ensuite stockées dans un répertoire pour une consultation facile.
2) **Enregistrement des résultats** : Les images fusionnées sont enregistrées sous forme de fichiers PNG dans les répertoires spécifiés (`TRAIN_PRED_PATH` et `VAL_PRED_PATH`).

3.5. Calcul du coefficient de Dice pour évaluer la performance du modèle

Le coefficient de Dice est une métrique couramment utilisée pour évaluer la précision de la segmentation. Il compare l'intersection des zones prédites et réelles (masques) par rapport à l'union des deux. Un coefficient de Dice élevé indique une bonne performance du modèle.

Code:

```
# Compute the average Dice coefficient for the training dataset
def dice_coefficient(y_true, y_pred):
    intersection = np.sum(y_true * y_pred)
    return (2. * intersection) / (np.sum(y_true) + np.sum(y_pred))

train_dice_scores = []
for i, frame in enumerate(train_frames):
    predicted_mask = m.predict(np.expand_dims(frame, axis=0))[0, :, :,
0]
    predicted_mask_binary = (predicted_mask > 0.5).astype(np.uint8)
    train_dice_scores.append(dice_coefficient(train_masks[i, :, :, 0],
predicted_mask_binary))
mean_dice_train = np.mean(train_dice_scores)

# Compute the average Dice coefficient for the validation dataset
val_dice_scores = []
for i, frame in enumerate(val_frames):
    predicted_mask = m.predict(np.expand_dims(frame, axis=0))[0, :, :,
0]
    predicted_mask_binary = (predicted_mask > 0.5).astype(np.uint8)
    val_dice_scores.append(dice_coefficient(val_masks[i, :, :, 0],
predicted_mask_binary))

# Affichage des resultas
plt.figure(figsize=(12, 6))
    plt.subplot(1, 3, 1)
    plt.title("Image originale")
    plt.imshow(frame[:, :, 0], cmap="gray")
    plt.subplot(1, 3, 2)
    plt.title("Masque prédit")
    plt.imshow(predicted_mask_binary, cmap="gray")
    plt.subplot(1, 3, 3)
    plt.title("Masque attendu (vérité terrain)")
```

```
    plt.imshow(val_masks[i, :, :, 0], cmap="gray")
    plt.show()
mean_dice_val = np.mean(val_dice_scores)
```

Cette partie permet d'afficher et d'évaluer les performances du modèle en calculant la moyenne du coefficient de Dice sur les images d'entraînement et de validation. Les résultats sont imprimés pour donner une indication claire de l'efficacité du modèle.

3.6. Calcul des distances Euclidienne et Haussdorff

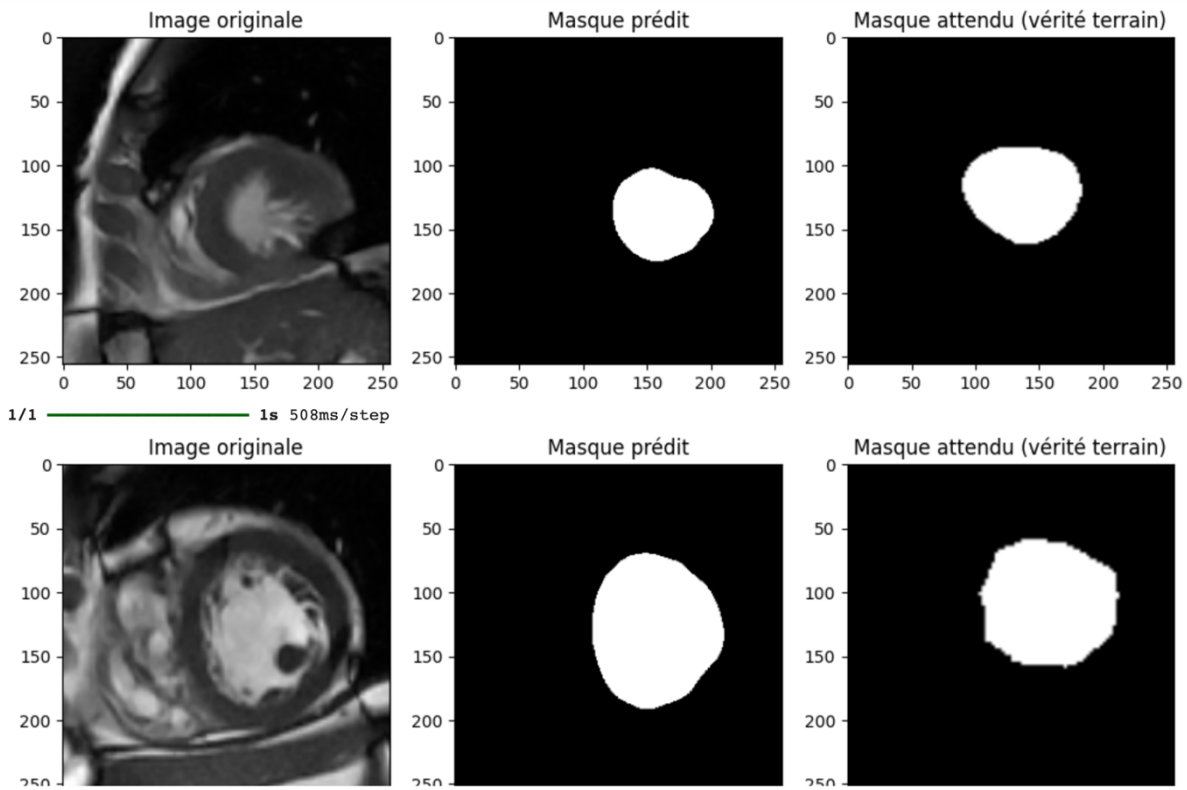
Les distances Euclidienne et Haussdorff sont également utilisées pour mesurer la similarité entre les contours des objets segmentés dans les images. Ces métriques sont particulièrement utiles pour évaluer la précision de la localisation des contours par rapport à la vérité terrain.

Code:

```
# Compute Euclidean and Haussdorff distances
def euclidean_distance(y_true, y_pred):
    """
    Calculate the mean Euclidean distance between points in the true
mask and the predicted mask.
    """
    true_points = np.argwhere(y_true)
    pred_points = np.argwhere(y_pred)
    distances = cdist(true_points, pred_points, metric='euclidean')
    return np.mean(np.min(distances, axis=1))

def hausdorff_distance(y_true, y_pred):
    true_points = np.argwhere(y_true)
    pred_points = np.argwhere(y_pred)
    forward_hd = directed_hausdorff(true_points, pred_points)[0]
    backward_hd = directed_hausdorff(pred_points, true_points)[0]
    return max(forward_hd, backward_hd)

mean_Euclidian_distance = np.mean([euclidean_distance(val_masks[i, :,
:, 0], (m.predict(np.expand_dims(frame, axis=0))[0, :, :, 0] >
0.5).astype(np.uint8)) for i, frame in enumerate(val_frames)])
mean_Hausdorff_distance = np.mean([hausdorff_distance(val_masks[i, :,
:, 0], (m.predict(np.expand_dims(frame, axis=0))[0, :, :, 0] >
0.5).astype(np.uint8)) for i, frame in enumerate(val_frames)])

print("Mean Dice coefficient for training database =
{}".format(mean_dice_train))
print("Mean Dice coefficient for validation database =
{}".format(mean_dice_val))
print("Mean Euclidian distance for validation database =
{}".format(mean_Euclidian_distance))
```

```
print("Mean Haussdorff distance for validation database =
{}".format(mean_Hausdorff_distance))
```

À l'issue de l'exécution de la cellule du code , les résultats obtenus sont les suivants :




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 461ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step
Mean Dice coefficient for training database = 0.5070593846418306
Mean Dice coefficient for validation database = 0.5496122513807951
Mean Euclidian distance for validation database = 6.843206923692645
Mean Haussdorff distance for validation database = 47.856535419920085
```

4. Conclusion

Ce projet a permis de mettre en œuvre un modèle de segmentation automatique des IRMs cardiaques en utilisant un réseau de neurones convolutionnel (CNN) de type U-Net. Après un prétraitement des données incluant normalisation et augmentation, le modèle a été entraîné avec succès sur des images IRM 2D pour segmenter l'endocarde. L'architecture U-Net, avec ses différentes couches d'encodeur et de décodeur, a permis d'obtenir des résultats de segmentation pertinents.

Les résultats du modèle ont été évalués à l'aide du coefficient Dice, une métrique clé pour la comparaison entre les masques prédits et les annotations manuelles. Grâce à des techniques telles que l'optimisation par la fonction de perte combinée (Dice Loss et Binary Crossentropy) et des callbacks comme EarlyStopping et ModelCheckpoint, le modèle a pu converger efficacement.

En conclusion, ce modèle montre qu'une approche de segmentation automatique par Deep Learning peut offrir des résultats prometteurs pour l'analyse d'IRM cardiaques, avec une réduction significative du temps et des ressources nécessaires par rapport aux méthodes manuelles.

5. Bibliographie :

[1] : DEEP LEARNING EN IMAGERIE MEDICALE Segmentation cardiaque en IRM , SARRY L.(déc. 2021)
[2] : https://en.wikipedia.org/wiki/U-Net. (déc. 2021)
[3] : https://www.mdpi.com/2072-4292/12/12/2001(déc. 2021)
[4] : https://towardsdatascience.com/understanding-semantic-segmentation-with-unet-6be4f42d4b47(déc. 2021)